



PROYECTO CYBERGUARD

Plataforma de análisis y detección de comportamientos anómalos en redes.



Corporación Unificada Nacional
de Educación Superior

VIGILADA MINEDUCACIÓN

Asignatura: Programación avanzada

Docente: Faber Alemán

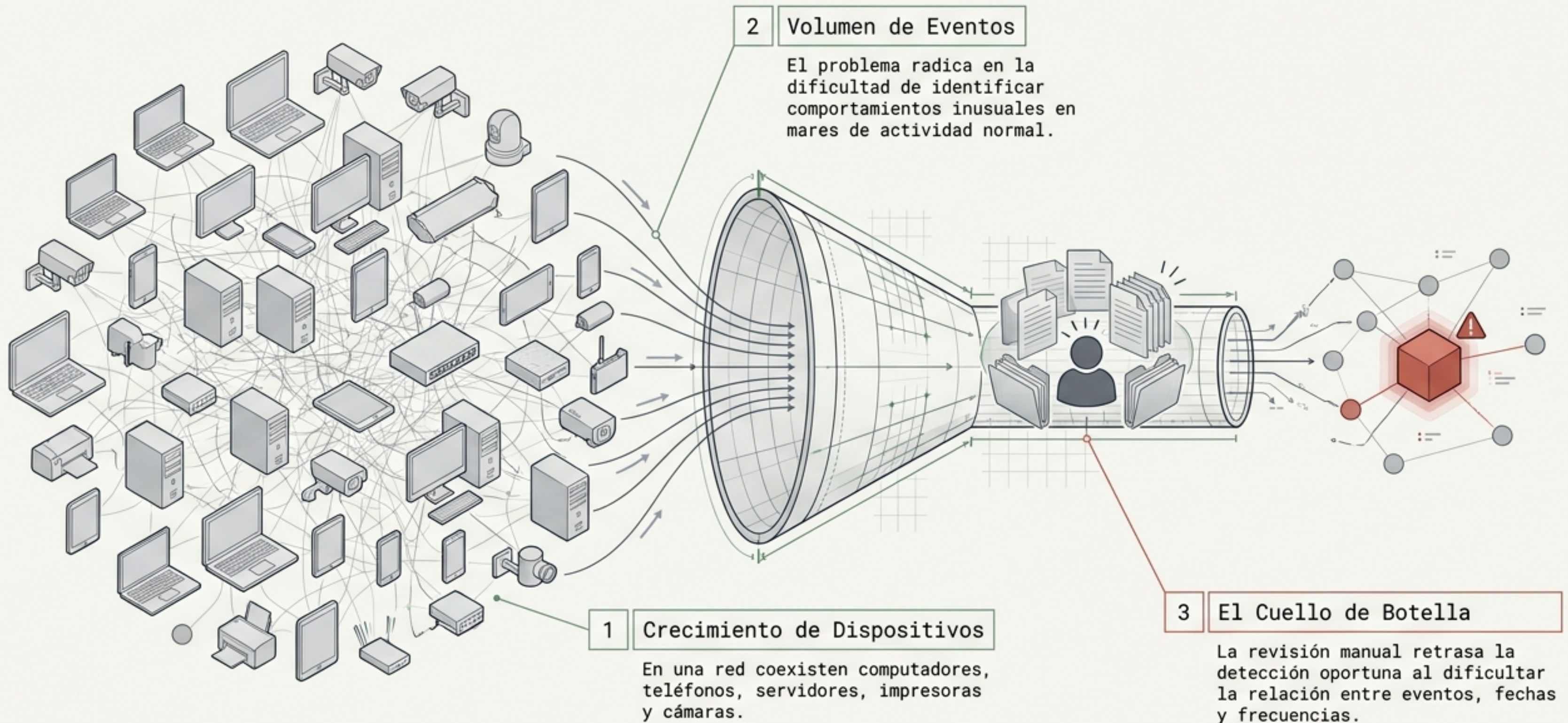
Programa: Ingeniería de Sistemas

Institución: CUN, Sincelejo, 2026

Equipo Desarrollador:

Sergio Esteban Veloza Gonzalez
Luifer Andres Hernandez Lambraño
Marcos Antonio Pacheco Bautista
Rafael Antonio Rodríguez Gamarra

El Laberinto de los Datos: El Problema de la Revisión Manual



La Solución: Análisis Automatizado Basado en Reglas

CyberGuard es una herramienta académica desarrollada en Java que centraliza la información de dispositivos y aplica un conjunto de reglas para identificar comportamientos anómalos de forma automática.



Registro Centralizado

Monitoreo estructurado de eventos dentro de redes autorizadas de laboratorio.

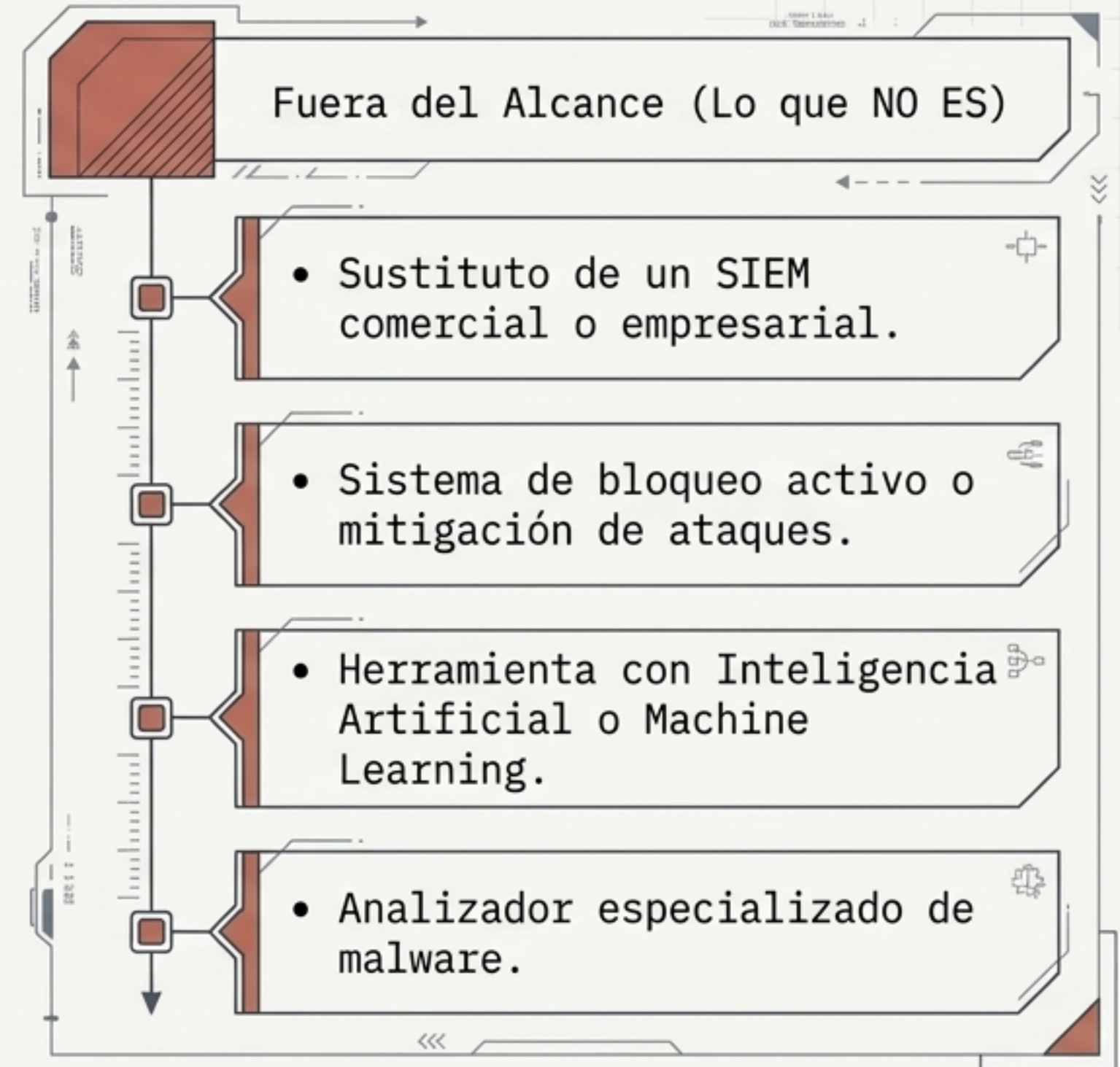
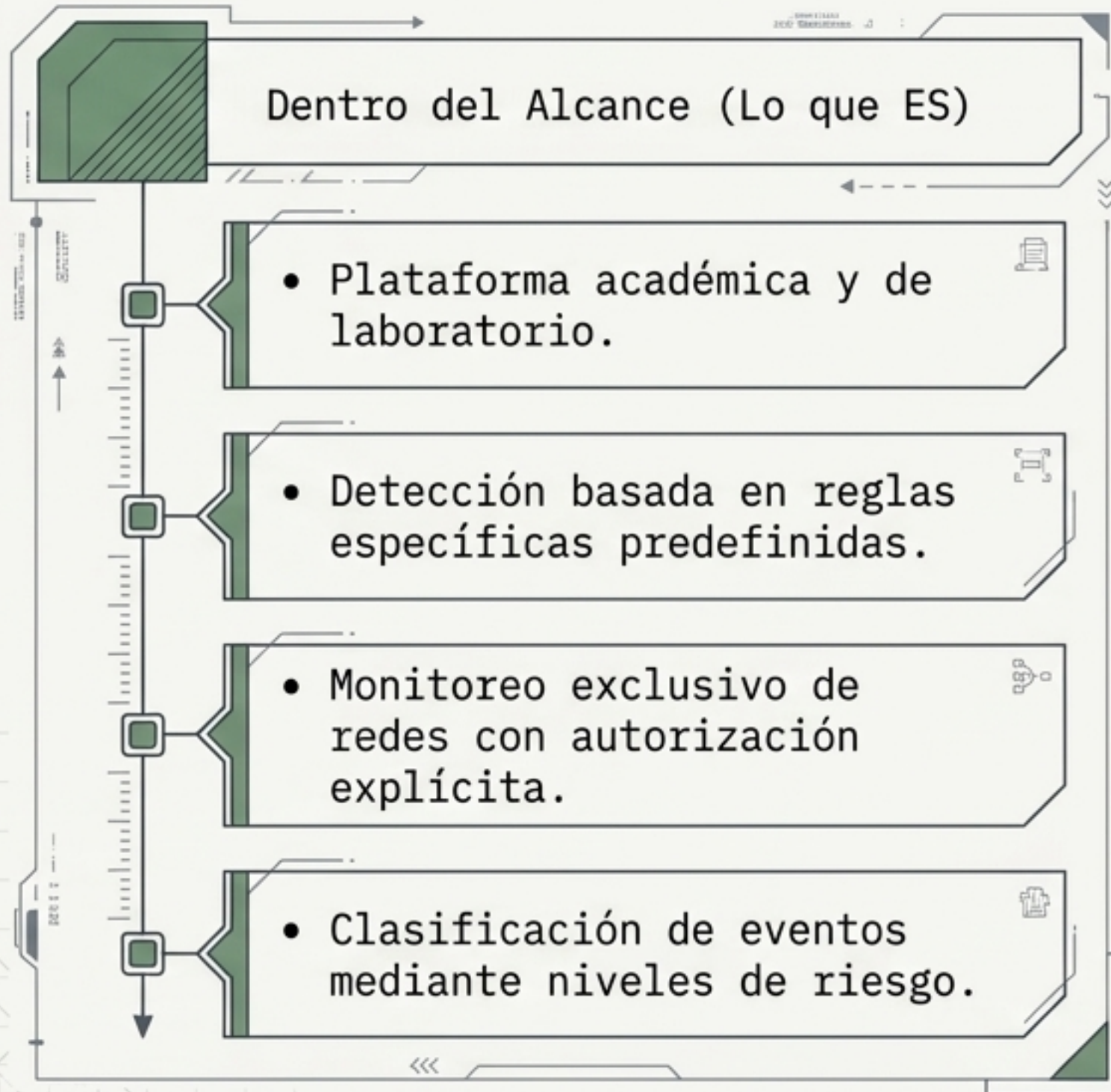
Análisis Inteligente

Procesamiento activo mediante reglas algorítmicas predefinidas, superando el simple almacenamiento.

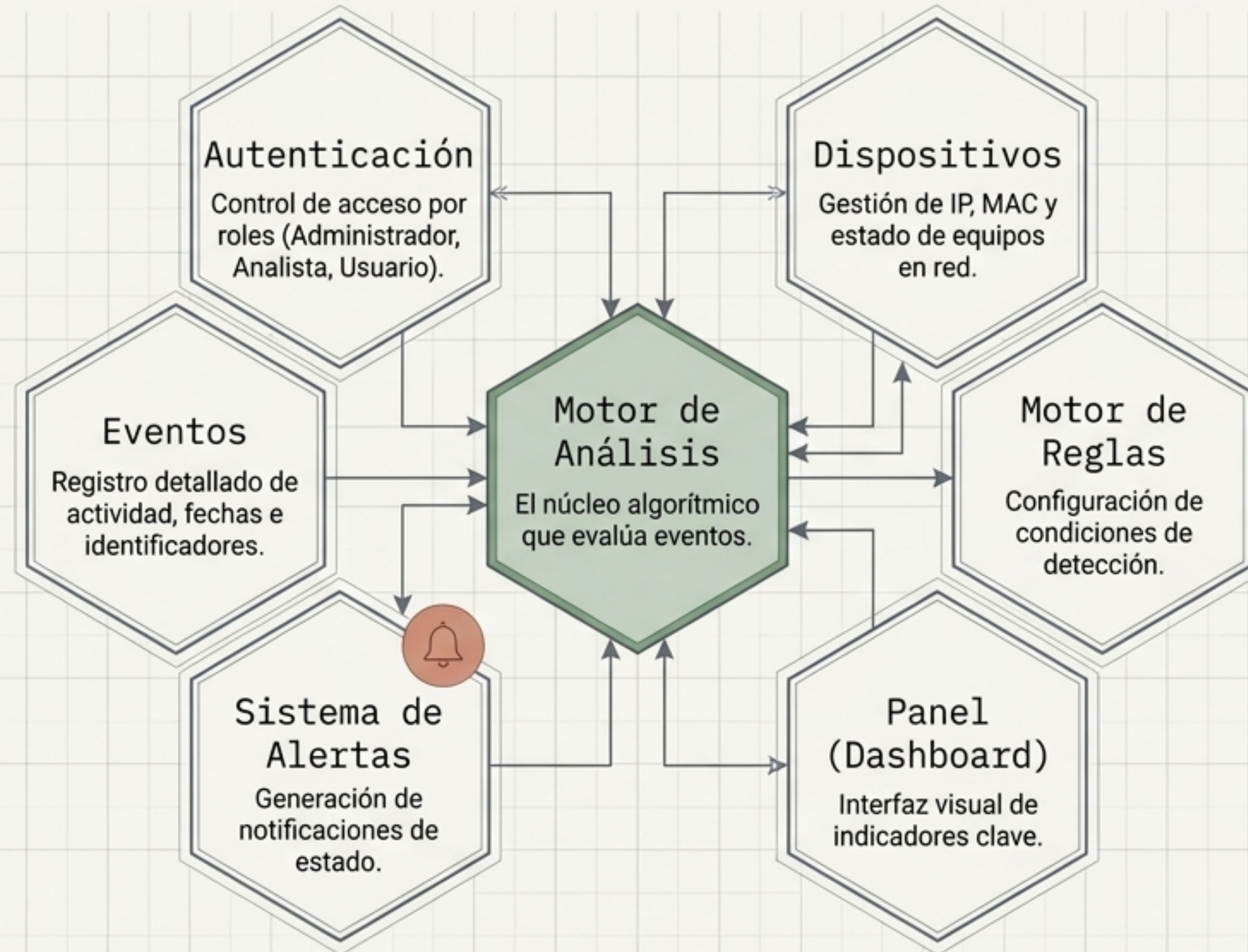
Alertas Oportunas

Clasificación técnica de riesgos para dirigir la atención del administrador a donde realmente importa.

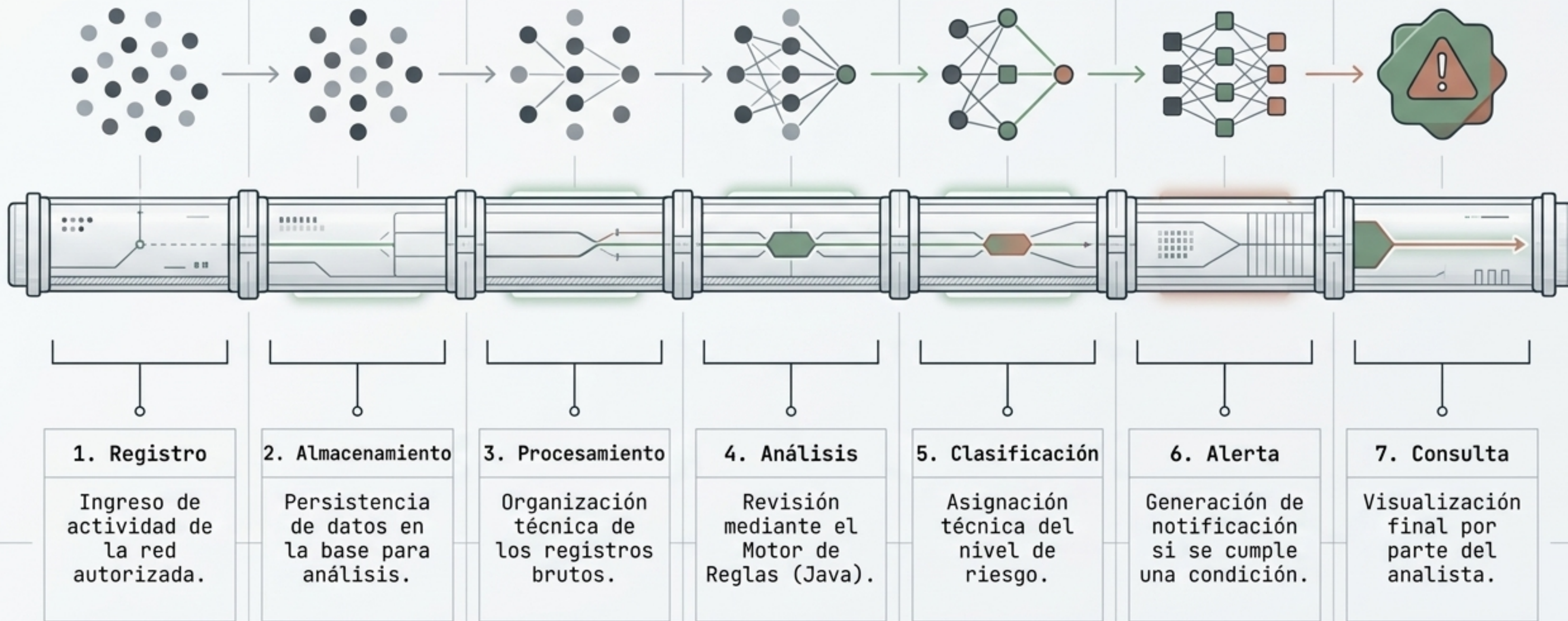
Alcance y Límites del Proyecto



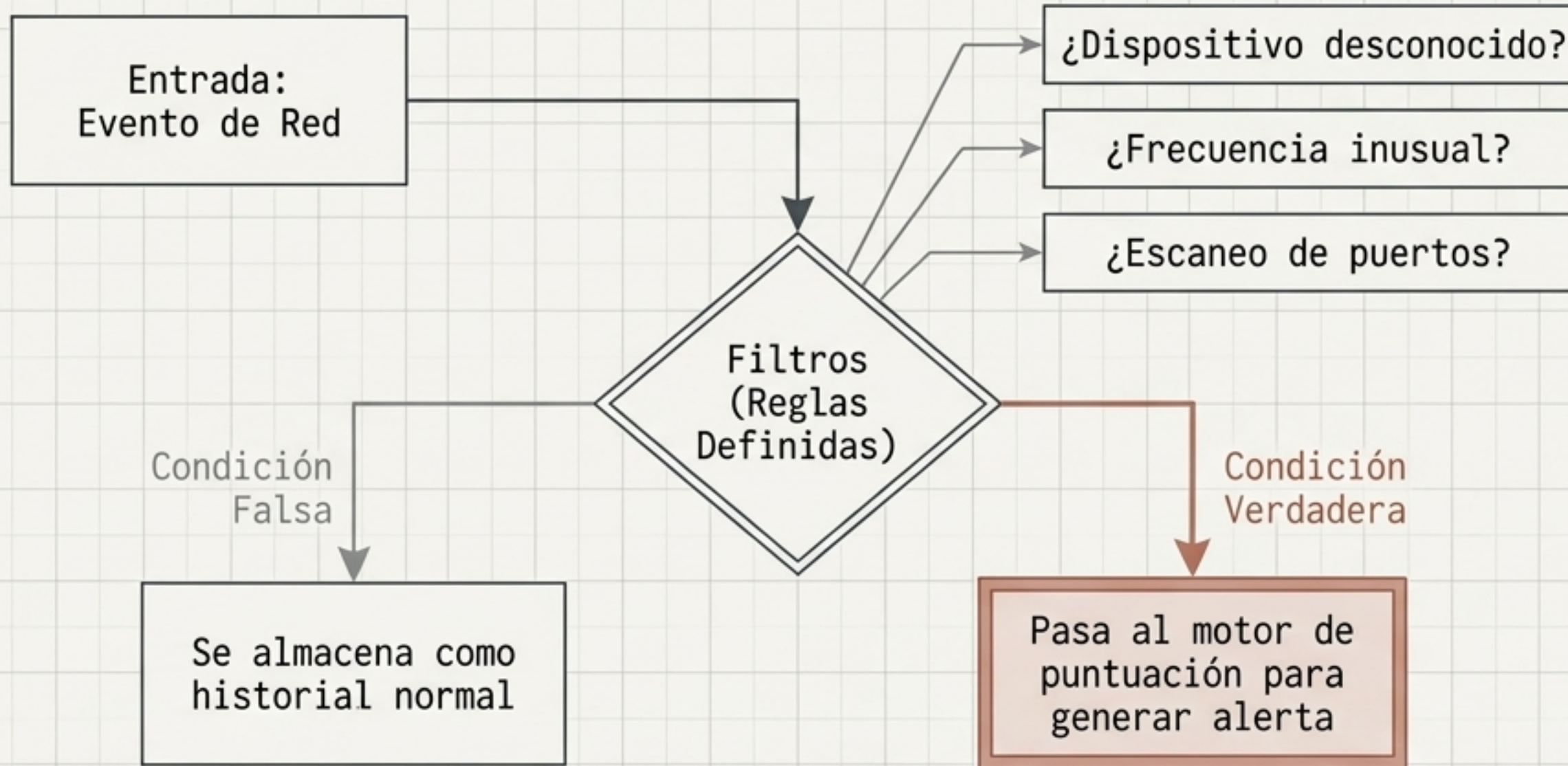
Ecosistema Modular de CyberGuard



Ciclo de Vida de un Evento en la Red



La Lógica del Motor de Análisis



Concepto Clave: Este componente requiere aplicar estructuras de datos y algoritmos de programación orientada a objetos (POO), pasando del simple almacenamiento a la toma de decisiones algorítmica.



Matriz Diagnóstica y Sistema de Puntuación

Disparadores (Triggers)

Dispositivo no registrado	-----	[+10 pts]
Actividad inusual	-----	[+10 pts]
Repetición de conexiones	-----	[+15 pts]
Múltiples intentos de conexión	----	[+20 pts]
Escaneo de diferentes puertos	----	[+25 pts]
Acumulación de eventos sospechosos	---	[+30 pts]

Termómetro de Riesgo



Crítico: 81–100 puntos

Alto: 51–80 puntos

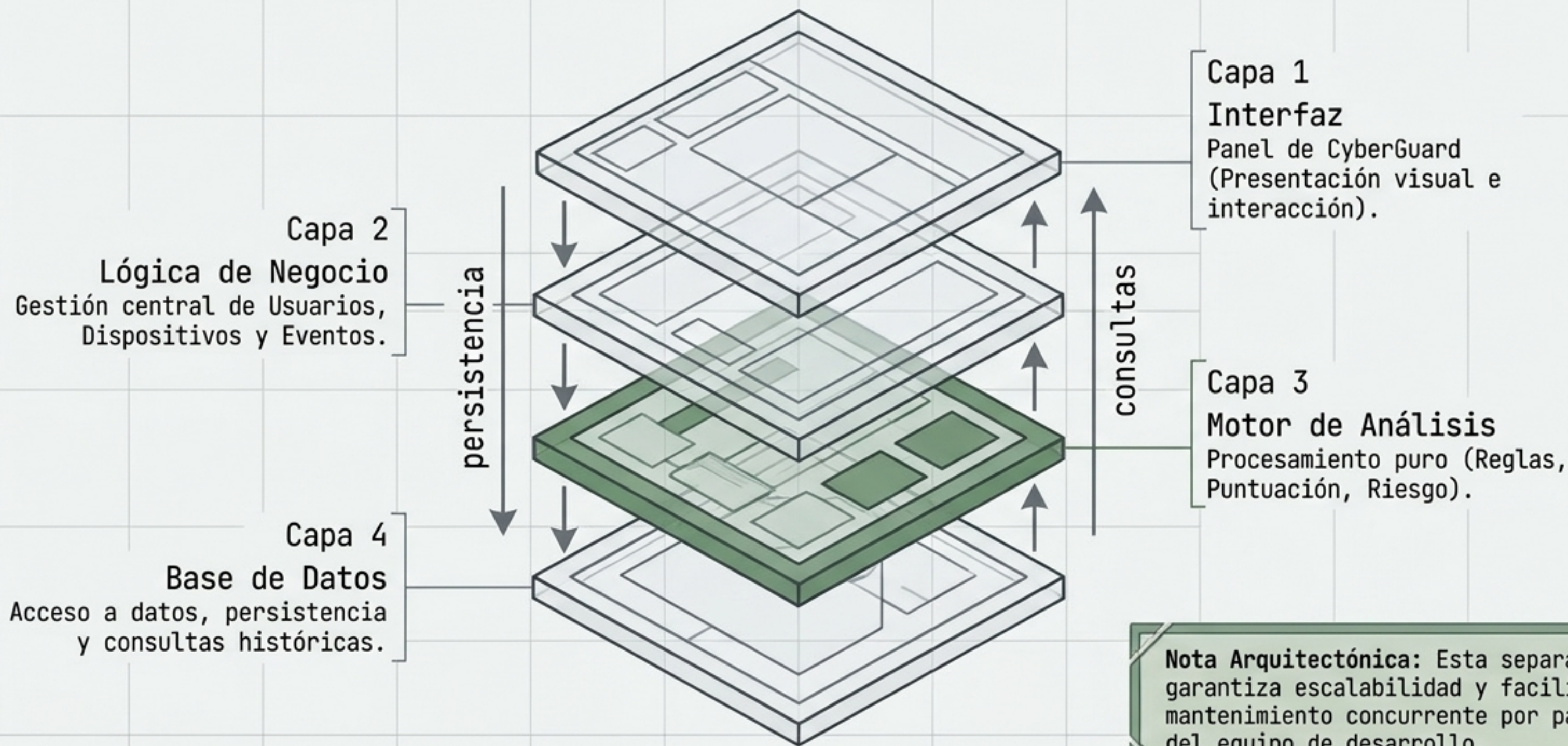
Medio: 21–50 puntos

Bajo: 0–20 puntos





Arquitectura de Software por Capas



Nota Arquitectónica: Esta separación garantiza escalabilidad y facilita el mantenimiento concurrente por parte del equipo de desarrollo.





Ecosistema Tecnológico



Backend

- Java (Lenguaje principal P00)
- Spring Boot (Framework)



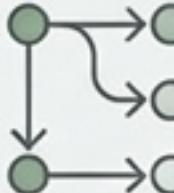
Base de Datos

- PostgreSQL / MySQL (Almacenamiento relacional)



Frontend

- HTML, CSS, JavaScript (Construcción del Dashboard)



Control de Versiones

- Git, GitHub (Integración de equipo)



Entornos (IDEs)

- IntelliJ IDEA, VS Code, Eclipse



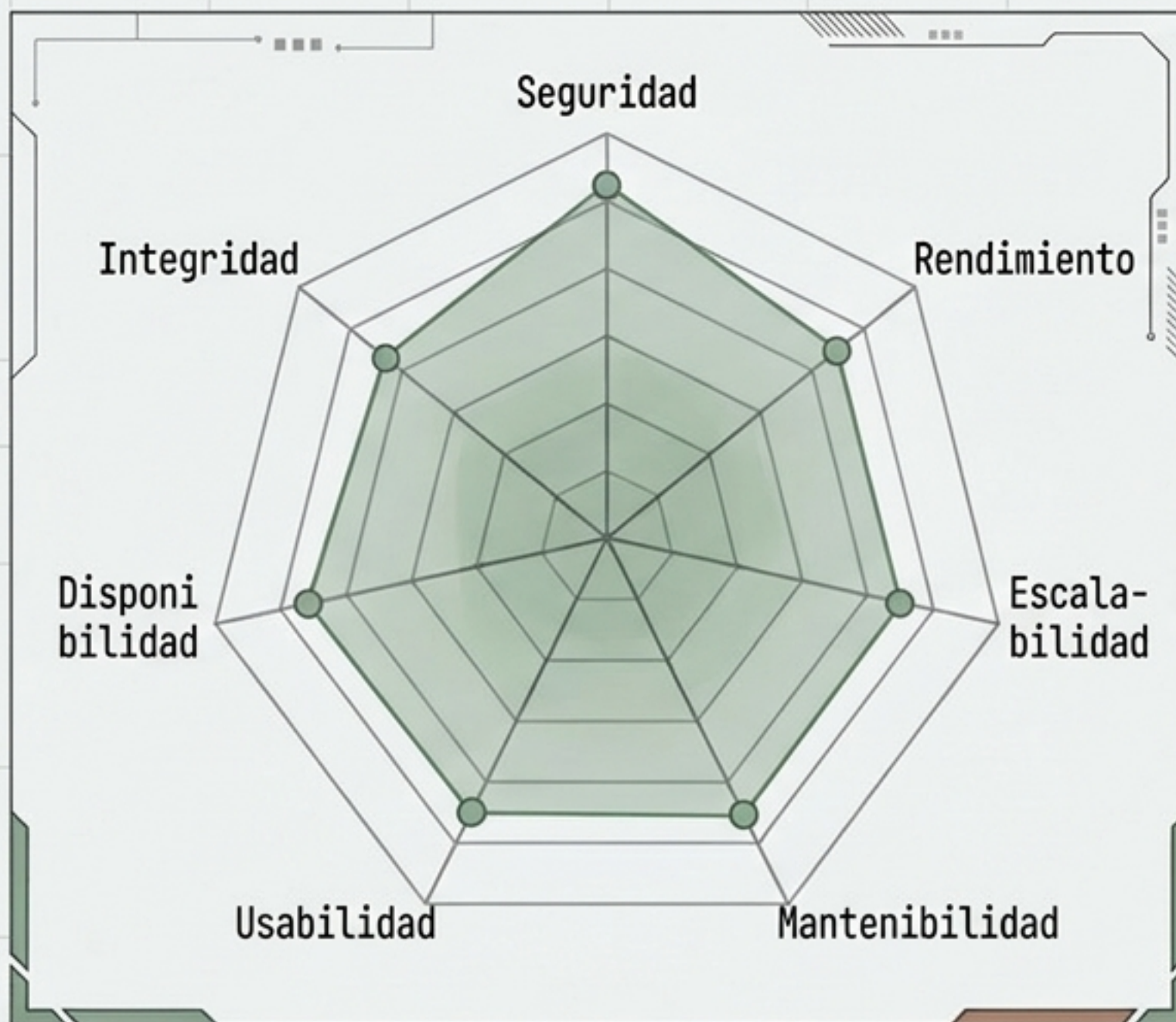
Herramientas de Laboratorio

- Nmap, Wireshark (Solo para simulación en redes autorizadas)





Requerimientos No Funcionales (Atributos de Calidad)



Seguridad

Control de acceso robusto y protección de credenciales.



Rendimiento

Procesamiento fluido sin cuellos de botella.



Escalabilidad

Crecimiento modular sin reconstruir el núcleo.



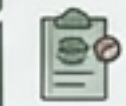
Mantenibilidad

Código limpio en componentes independientes.



Disponibilidad

Interfaz orientada a la rápida toma de decisiones.



Integridad

Consistencia relacional en la base de datos.





Matriz de Roles y Responsabilidades

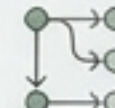
Sergio Esteban Veloza Gonzalez

Arquitectura principal y desarrollo general. Apoyo transversal en análisis e integración.



Luifer Andres Hernandez Lambraño

Lógica de negocio y procesamiento central de la información.



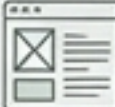
Marcos Antonio Pacheco Bautista

Diseño estructural de la Base de Datos y desarrollo del Motor de Reglas.



Rafael Antonio Rodríguez Gamarra

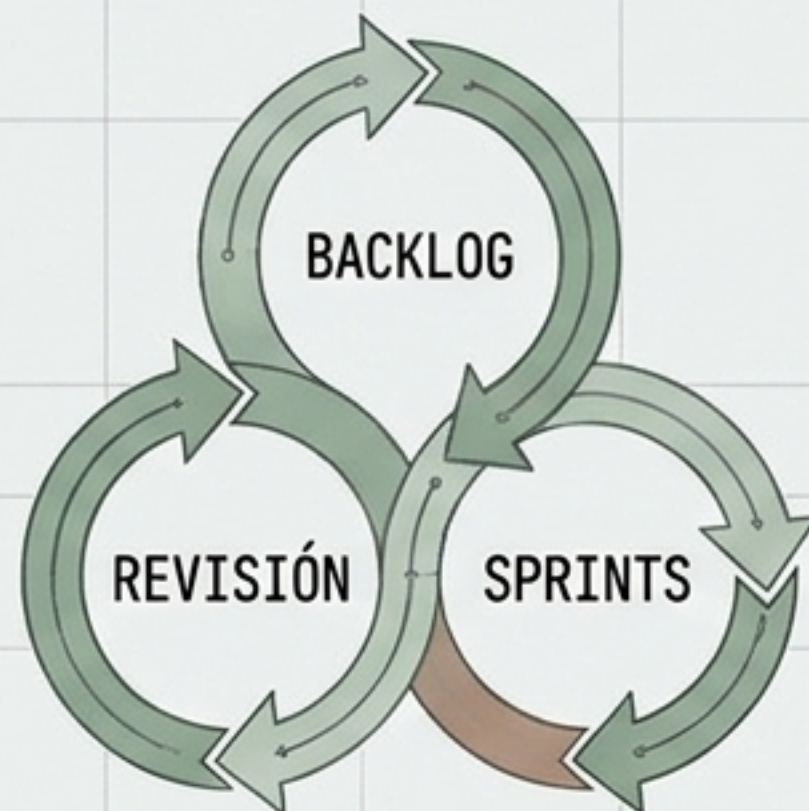
Diseño de Interfaz (Frontend) y apoyo vital en la integración visual de componentes.



Nota: Todos los integrantes participan de forma colaborativa en documentación, pruebas y análisis de requerimientos.



Metodología Scrum y Plan de Ejecución



Fase 1-2: Análisis de requerimientos y Diseño de arquitectura.

[MONOSPACE]
SCOPE DEFINITION.
SYSTEM BLUEPRINT.

Fase 4: Motor de análisis (Reglas y riesgos).

[MONOSPACE]
RULE ENGINE.
RISK ASSESSMENT LOGIC.

Fase 6: Integración del sistema.

[MONOSPACE]
COMPONENT INTEGRATION.
END-TO-END TESTING.

Fase 3: Desarrollo inicial (Usuarios y dispositivos).

[MONOSPACE]
USER MANAGEMENT.
DEVICE INTEGRATION.

Fase 5: Interfaz y Sistema de alertas.

[MONOSPACE]
UI/UX DEVELOPMENT.
ALERT NOTIFICATION
SYSTEM.

Fase 7-8: Pruebas, corrección de errores y Documentación final.

[MONOSPACE]
QUALITY ASSURANCE.
BUG FIXING.
FINAL DOCUMENTATION.



Visión Final: Ciberseguridad Académica Responsable



CyberGuard no es solo un repositorio de datos; es la aplicación práctica de la programación avanzada orientada a objetos para resolver un reto crítico en ciberseguridad. Estructura el caos de los datos, aplica lógica matemática para el diagnóstico, y provee una herramienta de laboratorio segura y controlada para la Universidad.

Gracias / Fin de la presentación
Proyecto de Ingeniería de Sistemas, 2026.